

Practical Task L2T1:

Question 1:

1. Layered Architecture: A traditional N-tier web application on Microsoft Azure

According to Microsoft, the N-tier architecture separates tasks like web or presentation, business logic and data access into logical and physical layers (Microsoft, n.d.).

This pattern is most suitable when developing business systems such as booking systems, banking systems, school management systems and internal company applications where roles must be clearly separated and the system may need to change over time. Microsoft points out that the N-tier architecture is helpful when requirements are still changing or when existing traditional systems are being transferred with minimal changes (Microsoft, n.d.).

This architecture is suitable because each layer has a clear responsibility which improves maintainability, testing, security and adaptability. For example, rules and logic are handled by the business layer, storage is managed by the data layer, and the user interface is handled by the presentation layer. If one part of the system changes, developers do not have to rewrite the whole system.

2. Repository Architecture: Amazon Redshift data warehouse

AWS describes Amazon Redshift as a fully managed, petabyte-scale data warehouse that allows users to add data and analyze it using BI and SQL tools. That means many tools and processes can work through one central data store (Amazon Web Services, n.d.).

This pattern is most suitable when an organization has large volumes of shared data that many separate components need to read from or write to, such as in analytics platforms, reporting systems, enterprise data platforms and hospital information systems.

The reason it fits with this example is that Redshift acts as a central data repository for reporting and analysis, while different external tools and services interact with that same store. This gives consistency and makes data easier to manage in one place. However, the central repository can become a single point of failure if not built or designed with strong resilience and backup.

3. Client-Server Architecture: PostgreSQL

PostgreSQL uses a client/server model, where the server manages the database and accepts connections, while client applications perform operations through that server (PostgreSQL Documentation, 2023).

This pattern is most suitable when multiple users or applications need to access shared services over a network, such as in database systems, email systems, web applications and file servers. It works well in situations whereby multi-user access, shared resources, and centralized control are required.

The reason it suits PostgreSQL is because the client and server have clear separate roles. The client sends requests, and the server processes those requests and returns results.

4. Pipe and Filter architecture: Logstash Pipelines in the Elastic stack

Logstash has real-time pipelining capabilities and can use input, filter and output stages to unify and transform data before sending it to destinations (Elastic.co, 2025).

This architecture is most suitable for data processing, log processing, ETL workflows, event streaming and message transformation where data must pass through several stages before reaching the final output.

The reason this architecture fits Logstash is that incoming data is collected, transformed, cleaned and forwarded step by step. Each stage performs a specific task, which is exactly how a filter chain works. This makes the system flexible because filters can be added, removed or changed without redesigning the whole flow. The disadvantage is that processing time and cost may rise with the number of transformation stages.

Question 2:

1. Azure N-tier web application

A modern azure web application can combine layered architecture and client-server architecture. According to Microsoft's architecture guidelines, a client communicates data levels between web and business tiers. As clients send requests to servers, the system functions as a client-server system and it also works as layered because responsibilities are separated into layers (Microsoft, n.d.).

The strength of this combined system is that it gives both good structure and shared access. The layered side improves security measures, maintainability and easier testing. The client-server side also allows many users to access the application over a network. It is therefore very suitable for systems like student portals, e-commerce websites, business dashboards and banking apps.

Its limitation is that it can become more difficult to deploy and manage as more layers are added. Additionally, Microsoft points out that while physical separation between layers improves scalability and resilience, it also increases delay due to

additional network traffic and as a result the system becomes strong but also heavier and more complicated (Microsoft, n.d.).

2. **Elastic stack:**

The Elastic Stack is a good example of a system combining pipe and filter architecture with repository architecture. Elastic explains that the stack includes Logstash, Elasticsearch and Kibana, and that it takes data from many sources so it can be searched, analyzed and visualized. Logstash handles pipeline processing while Elasticsearch acts as the central indexed data store (Elastic.co, 2025).

The strength of this combined system is that it handles data very effectively. The pipe and filter side makes it easy to clean, transform and improve incoming logs or events. The repository side provides a central location where processed data can be stored and queried by dashboards and analytics tools. This makes the stack very useful for security monitoring and log analytics.

Its limitation is that the system can become resource-intensive and harder to manage at scale. The more filters and transformations you add, the more processing power and tuning are required. Also, because so much value depends on the central store, problems in the storage or indexing layer can affect the usefulness of the whole platform (Elastic.co, 2025).

References List

docs.aws.amazon.com. (n.d.). *Amazon Redshift system overview - Amazon Redshift*. [online] Available at: <https://docs.aws.amazon.com/redshift/latest/dg/welcome.html>.

Elastic.co. (2025). *Logstash | Elastic Documentation*. [online] Available at: <https://www.elastic.co/docs/reference/logstash>.

Microsoft (n.d.). *N-tier architecture style - Azure Architecture Center*. [Online] learn.microsoft.com. Available at: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier>.

PostgreSQL Documentation. (2023). *1.2. Architectural Fundamentals*. [online] Available at: <https://www.postgresql.org/docs/current/tutorial-arch.html>.